

Coral-Chords documentation

Linus Tibert

August 7, 2025

Contents

1	Used abbreviations	3
2	Introduction	4
3	Project structure	4
3.1	Backend	4
3.1.1	UG scraper module	4
3.1.2	System module	4
3.1.3	Types module	4
3.2	Frontend	5
4	Functionality details	5
4.1	Getting the data from UG	5
4.1.1	Legal aspects of web scraping	5
4.1.2	Scraping UG's data	5
4.2	Storing the data	6
4.3	Interaction with Spotify	6

1 Used abbreviations

Although some of those might seem obvious, I'll give a complete overview of special abbreviations used in the document for those who are not familiar with any of them.

CCh	Coral-Chords
UG	Ultimate Guitar
BE	backend
FE	frontend
UI/GUI	(graphical) user-interface
URL	uniform resource locator (web-link)
API	application programming interface
ID/UID	(unique) identifier

2 Introduction

This document is supposed to be a useful guide for those who want to understand how the code of CCh works. It is especially designed to be an explanation of how each component of the program is working, however, some detailed parts (such as specific `functions()`) are not mentioned here, because they are easier to understand (if you know what the whole section of the code is for), when looking at the actual code.

To make this document more accessible for everyone, it is divided into many small sections, each with a clear subject.

3 Project structure

The project is divided into two major parts: the backend (referred to as BA), and the main program (called the frontend or FA).

The FE is the part of the program which interacts directly with the user and which is also creating the UI.

The BA is where the magic happens. Its purpose is to do anything not directly involving the user. Thus, it fulfils tasks involving networking, data extraction or direct interaction with the (file) system.

3.1 Backend

Because the BA still has many tasks, it is subdivided into a bunch of sub modules defined in the code. I'll explain the tasks of each of them in the following sections.

3.1.1 UG scraper module

This module is used to retrieve data from UG and process it into useable data.

This module was originally developed as a local library just for this project, but after it was finished, it got uploaded publicly as a separate crate to crates.io and GitHub.¹

3.1.2 System module

As the name suggests, every direct interaction with the user's system is managed by this module. It is mainly used to read and write files and for detection of the currently running song on Spotify.

3.1.3 Types module

All globally used types and constants are defined here.

This module mainly exists to keep the code clean. Having all custom types in one module makes it easier to manage them across all parts of the project.

¹ <https://github.com/Lich-Corals/ug-scraper-rs/>

3.2 Frontend

4 Functionality details

4.1 Getting the data from UG

Ultimate Guitar does not have an API a publicly available API. I.e., there is no easy way for developers outside the company to use their site as a data source. Thus, the best way to get UG's data is to get their source HTML and run a couple of RegEx patterns on it to get raw data.

This is called “web scraping” and is exactly what CCh, or the `ug_scraper` library does. This way CCh gets a very limited kind of API to get machine processable data from UG. In the following sections I will elucidate the way CCh scrapes the data from raw HTML on UG.

4.1.1 Legal aspects of web scraping

First, a tiny disclaimer: Web scraping is not illegal, and in this case it is legal too.

The important thing is that CCh is not downloading/scraping any restricted or confidential data. All it really does is downloading the same contents your web browser is downloading as soon as you look at a tab or search result at UG.

Besides this, CCh does not contain any scraped (i.e., copyrighted) data; with publishing this project, I only supply people with an easy method of getting UG's tabs in a custom UI without re-publishing any of the copyrighted materials of UG.

To read more about this topic, you can visit my source of those facts in the footnote.²

4.1.2 Scraping UG's data

Everything explained in this section is happening in the `ug_scraper` crate. Thus, if you want a more technical view of it, I recommend taking a look at the crate at crates.io or GitHub.

What the crate is doing to get data from UG can essentially be described with three steps:

1. Download data from UG
2. Extract valuable information
3. Output data in a custom format

In the first step, a simple function is triggered which returns the raw HTML of the selected UG page. (This page is either a tab or a search results page.) To make sure the downloaded data is valid for data extraction, the program simply checks for a few strings within the HTML; if certain ones are present and certain ones are not present, the page is valid.

The data extraction is done with a few RegEx patterns. Those are basically collections of special characters telling the computer to look for a specific pattern in

² <https://blog.apify.com/is-web-scraping-legal/>

a given piece of data. Within those patterns, certain groups are defined, telling the RegEx engine which parts of the pattern are the wanted data.

As soon as the data is extracted, it is saved

4.2 Storing the data

4.3 Interaction with Spotify